BSP Registry Tools

Streamlined Yocto & Isar BSP Management
for Embedded Linux Teams

From registry to built image — one command

```
pip install bsp-registry-tools
github.com/Advantech-EECC/bsp-registry
License:  Apache 2.0
```

Contents

1. Overview

BSP Registry Tools is a Python CLI and library for managing Board Support Packages (BSPs) for embedded Linux systems based on Yocto Project or Isar (Debian-based). It uses a YAML registry (schema v2.0) as the single source of truth for all configuration, enabling reproducible, team-shareable builds across dozens of boards and release combinations.

- Python CLI tool (`bsp`) and importable Python API (`BspManager`)
- YAML registry v2.0 — devices, releases, features, presets, environments
- Wraps KAS (`kas` / `kas-container`) for Yocto and Isar builds
- Zero-config first run — auto-clones the Advantech registry on first use
- Named remote management — save/reuse remote registries like git remotes
- Environment variable expansion via `$ENV{VAR}` in registry YAML
- Cloud artifact deploy / gather (Azure Blob Storage, AWS S3)
- HTTP server with REST + GraphQL APIs (FastAPI + Strawberry)
- Shell tab completions for all arguments (Bash, Zsh, Fish, tcsh)
- HIL test triggering via LAVA with Robot Framework suites
- Interactive registry editor (`bsp registry add/edit/remove`)
- Registry validation, diff, and split across multiple files

2. System Architecture

Figure **??** shows how the BSP Registry Tools components interact. The `BspManager` API is the central hub connecting all user-facing interfaces (CLI, TUI/GUI explorer, HTTP server) to the underlying build infrastructure (KAS, container engines, cloud storage, LAVA test infrastructure).

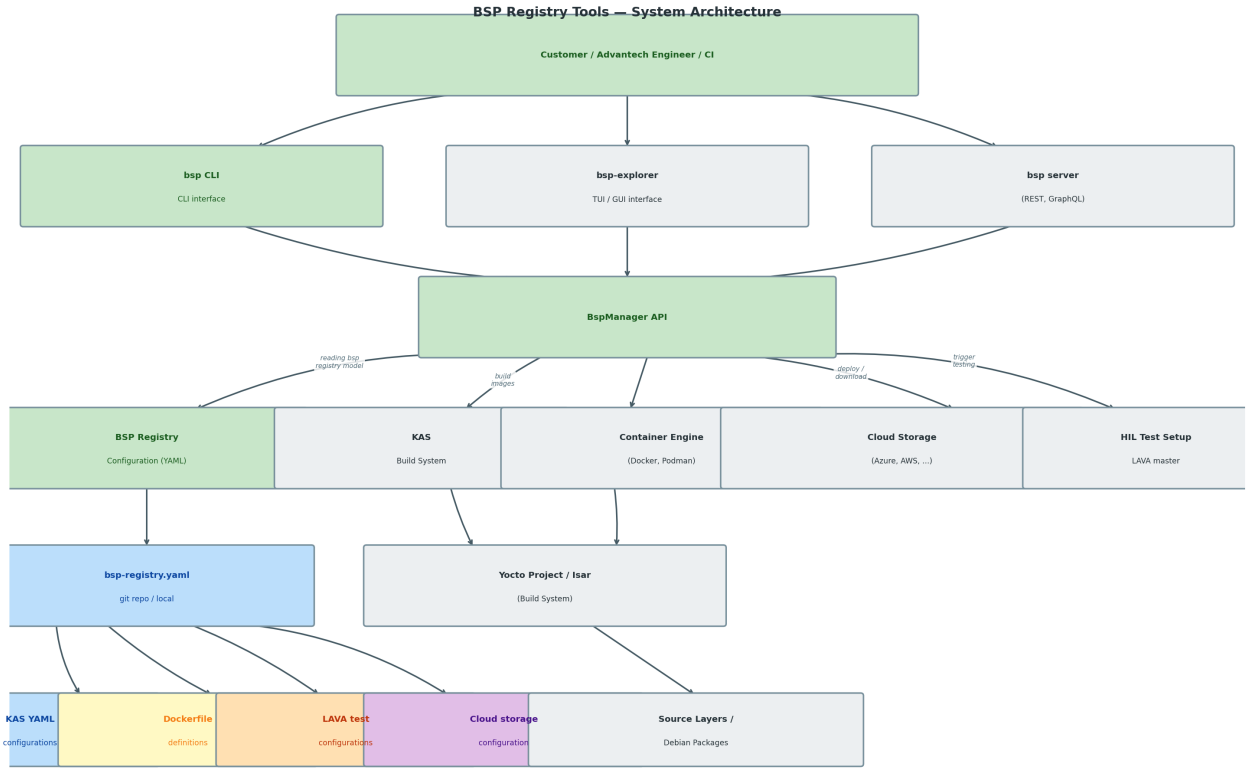

Figure 1: BSP Registry Tools — System Architecture

2.1 Component Roles

Component	Role
bsp CLI	Primary user interface — build, list, export, deploy, gather, registry CRUD
bsp-explorer	TUI / GUI interface for interactive registry browsing
bsp server	HTTP REST + GraphQL server for automation and dashboards
BspManager API	Core Python library coordinating all operations
BSP Registry YAML	Single source of truth: devices, releases, features, presets
KAS Build System	Orchestrates Yocto BitBake or Isar builds from YAML config files
Container Engine	Docker / Podman for isolated, reproducible build environments
Cloud Storage	Azure Blob / AWS S3 for artifact upload and download
HIL Test / LAVA	Hardware-in-the-loop test automation via LAVA
bsp-registry.yaml	Git repository or local folder holding all KAS YAML, Dockerfiles, LAVA test configs, and cloud storage config

3. Installation

Python 3.8 or later is required. A virtual environment is strongly recommended.

3.1 From PyPI

```
pip install bsp-registry-tools
```

3.2 From Source

```
git clone https://github.com/Advantech-EECC/bsp-registry-tools.git
cd bsp-registry-tools
pip install .
```

3.3 Optional extras

Extra	Installs
`[azure]`	Azure Blob Storage support (`azure-storage-blob`, `azure-identity`)
`[aws]`	AWS S3 support (`boto3`)
`[deploy]`	Both Azure and AWS
`[server]`	FastAPI + uvicorn + Strawberry GraphQL
`[completions]`	Shell tab completions (`argcomplete`)
`[dev]`	pytest, coverage, ruff, mypy

```
pip install "bsp-registry-tools[azure,completions,server]"
```

3.4 Core Dependencies

Package	Purpose
PyYAML >= 6.0	YAML registry parsing
dacite >= 1.6.0	Dataclass deserialization from YAML
kas >= 4.7	KAS build orchestrator
colorama >= 0.4.6	Coloured terminal output
requests >= 2.28.0	LAVA HIL test integration
Jinja2 >= 3.1.0	LAVA job template rendering

4. Supported Hardware

4.1 NXP i.MX Boards (Advantech Europe)

Board	SoC	Yocto Releases	Status
RSB-3720	i.MX8M Plus	scarthgap, styhead, walnascar, whinlatter	Stable
RSB-3720 4G	i.MX8M Plus	walnascar, whinlatter	Stable
RSB-3720 6G	i.MX8M Plus	walnascar, whinlatter	Stable
ROM-2620	i.MX8	scarthgap, styhead, walnascar, whinlatter	Stable
ROM-2820	i.MX93	scarthgap, styhead, walnascar, whinlatter	Stable
ROM-5720	i.MX8	scarthgap, styhead, walnascar, whinlatter	Stable
ROM-5721	i.MX8	scarthgap, styhead, walnascar, whinlatter	Stable
ROM-5722	i.MX8	scarthgap, styhead, walnascar, whinlatter	Stable
AOM-5521	i.MX95	scarthgap, walnascar	Stable

4.2 MediaTek & Qualcomm Boards

Board	SoC	Releases	Status
RSB-3810	MediaTek MT8395	scarthgap	Development
Genio 1200 EVK	MediaTek MT8395	scarthgap	Development
AOM-2721	Qualcomm QCS6490	scarthgap	Development
QCS6490 RB3 Gen2	Qualcomm QCS6490	scarthgap	Development

4.3 Emulated Targets (QEMU)

- `qemuarm64` — 64-bit ARM (AArch64)
- `qemuarm` — 32-bit ARM
- `qemux86` — x86 32-bit
- `qemux86-64` / `qemuamd64` — x86-64

5. Supported Releases

5.1 Yocto Releases

Slug	Yocto Version	LTS	Notes
kirkstone	4.0	LTS	poky, fsl-imx-xwayland
mickledore	4.2		poky, fsl-imx-xwayland
nanbield	4.3		poky
scarthgap	5.0	LTS	poky, fsl-imx-xwayland, mediatek, qualcomm, ros2
styhead	5.1		poky, fsl-imx-xwayland
walnascar	5.2		poky, fsl-imx-xwayland
whinlatter	5.3		poky, fsl-imx-xwayland
wrynose	6.0		poky (upcoming)

5.2 Isar (Debian-based) Releases

Slug	Distribution	Base Image
debian-trixie	Debian 13 (Trixie)	isar-debian-13 container
ubuntu-noble	Ubuntu 24.04 LTS	isar-debian-13 container
ubuntu-jammy	Ubuntu 22.04 LTS	isar-debian-13 container

6. Registry Schema v2

The registry uses a YAML file (default: `bsp-registry.yml`) with schema version 2.0. The top-level structure is:

```yaml
specification:
  version: "2.0"           # required; tool exits on mismatch

include:                   # optional -- list of additional YAML files to
    merge
  - devices/boards.yaml
  - releases/scarthgap.yaml

environment:               # optional -- global env vars for every build
  variables:
    - name: "DL_DIR"
      value: "$ENV{HOME}/downloads"
  copy:                    # optional -- files copied before every build
    - scripts/global-setup.sh: build/

environments:              # optional -- named build environments (dict)
  default:
    container: "ubuntu-22.04"
    variables:
      - name: "SSTATE_DIR"
```

```
        value: "$ENV{HOME}/sstate"

containers:                # optional -- Docker image definitions (dict)
  ubuntu-22.04:
    image: "bsp/kas:5.2"
    file: Dockerfile

registry:
  frameworks: [...]        # build-system framework defs (Yocto, Isar)
  distro: [...]            # distribution defs (Poky, fsl-imx-xwayland,
    ...)
  vendors: [...]           # cross-release vendor KAS fragments
  devices: [...]           # hardware board definitions
  releases: [...]          # Yocto / Isar release definitions
  features: [...]          # optional add-ons
  bsp: [...]               # named presets (device + release + features)

deploy:                    # optional -- cloud artifact upload config
  provider: azure          # "azure" (default) or "aws"
  container: bsp-artifacts
  prefix: "{vendor}/{device}/{release}/{date}"
```

6.1 Registry Sections Summary

Section	Purpose
`frameworks`	Build-system definitions (Yocto, Isar); referenced by distros
`distro`	Distribution definitions (Poky, fsl-imx-xwayland, ...)
`vendors`	Cross-release board-vendor KAS fragments
`devices`	Hardware board definitions (slug, vendor, SoC, KAS includes)
`releases`	Yocto/Isar release definitions with optional vendor overrides
`features`	Optional composable add-ons (OTA, secure-boot, ROS2, ...)
`bsp`	Named presets = device + release(s) + features
`environments`	Container + variable bundles per build class
`containers`	Docker image definitions with optional volume mounts
`deploy`	Cloud artifact upload configuration (Azure / AWS)
`include`	Split large registries across multiple files

6.2 File Splitting with `include`

Large registries can be split across multiple YAML files using the top-level `include` directive. Each included file is merged before the root file's own content is processed.

Data type	Merge behaviour
Lists (devices, releases, ...)	Concatenated — included items appear before the including file's items
Dicts (containers, environments)	Merged recursively — including file wins on conflicting keys
Scalars	Including file wins

Included files can themselves contain `include` directives (paths are always resolved relative to the file containing the directive). Circular includes are detected at load time. The `specification` block is silently stripped from included files.

6.3 Device Definition

```
registry:
  devices:
    - slug: rsb3720           # unique identifier
      description: "Advantech RSB-3720 (i.MX8M Plus, 6GB)"
      vendor: advantech-europe
      soc_vendor: nxp
      soc_family: imx8        # optional
      includes:
        - vendors/advantech-europe/nxp/machine/imx8/rsb3720.yml
      local_conf:             # optional: extra lines appended to local.
        conf
        - "MACHINE_EXTRA_RDEPENDS += 'kernel-modules'"
      copy:                   # optional: files copied before the build
        - scripts/setup.sh: build/rsb3720/
```

Field	Type	Description
`slug`	string	Unique identifier; used in CLI and presets
`vendor`	string	Board vendor name
`soc_vendor`	string	Silicon vendor (for compatibility checks)
`soc_family`	string?	SoC family (for compatibility checks)
`includes`	list	Device-specific KAS configuration files
`local_conf`	list	Lines appended to `local.conf`
`copy`	list	Files to copy before each build

6.4 Release Definition

```
registry:
  releases:
    - slug: scarthgap
      description: "Yocto 5.0 LTS (Scarthgap)"
      yocto_version: "5.0"
      distro: poky              # references distro[*].slug
      # environment: default    # optional named environment
      includes:
        - kas/scarthgap.yaml
```

```
    vendor_overrides:          # optional per-vendor overrides (see next
        section)
      - vendor: advantech-europe
        includes:
          - kas/advantech-europe/scarthgap-vendor.yaml
```

6.5 Named Preset (BSP)

```
registry:
  bsp:
    - name: modular-bsp-rsb3720
      description: "Advantech RSB-3720 (i.MX8M Plus)"
      device: rsb3720
      releases: [scarthgap, styhead, walnascar, whinlatter]
      features: [systemd, security, virtualization, ipv6, usrmerge]
      build:                        # optional overrides
        path: build/modular-bsp-rsb3720
        # container: "ubuntu-22.04" # optional container override
    - name: modular-bsp-rsb3720-imx6.6.53
      description: "RSB-3720 pinned to i.MX BSP 6.6.53"
      device: rsb3720
      release: scarthgap            # single-release form
      vendor_release: imx-6.6.53    # pin kernel/firmware version
      features: [systemd, security]
```

6.6 Vendor Overrides

Vendor overrides let a single release slug (e.g. `scarthgap`) apply different KAS fragments, distro settings, and kernel/firmware version pins per hardware vendor and SoC family.

6.6.1 Basic Vendor Override

```
releases:
  - slug: scarthgap
    distro: poky
    includes:
      - kas/scarthgap.yaml
    vendor_overrides:
      - vendor: advantech-europe
        distro: fsl-imx-xwayland   # overrides release.distro for this
            vendor
        includes:
          - kas/advantech-europe/scarthgap-vendor.yaml
        releases:                  # optional kernel/firmware sub-
            releases
          - slug: imx-6.6.52-2.2.2
            includes:
              - kas/advantech-europe/nxp/imx-6.6.52-2.2.2-scarthgap.yml
          - slug: imx-6.12.0
            includes:
              - kas/advantech-europe/nxp/imx-6.12.0-scarthgap.yml
```

6.6.2 SoC Vendor Overrides

When a board vendor ships hardware based on multiple SoC families, the `soc_vendors` list selects per-SoC-family KAS fragments automatically by matching the device's `soc_vendor` field:

```
vendor_overrides:
  - vendor: advantech
    includes:
      - kas/advantech/scarthgap-common.yaml  # common for all Advantech
          boards
    soc_vendors:
      - vendor: nxp
        distro: fsl-imx-xwayland
        includes:
          - kas/advantech/nxp/scarthgap.yaml
        releases:
          - slug: imx-6.6.53
            includes: [kas/advantech/nxp/imx-6.6.53.yaml]
          - slug: imx-6.12.0
            includes: [kas/advantech/nxp/imx-6.12.0.yaml]
      - vendor: mediatek
        distro: mt-distro
        includes:
          - kas/advantech/mediatek/scarthgap.yaml
```

6.6.3 KAS Include Ordering

The resolver assembles the final KAS file list in this order:

1. `framework.includes`
2. `distro.includes` (uses `SocVendorOverride.distro` > `VendorOverride.distro` > `Release.distro`)
3. `vendors[device.vendor].includes`
4. `release.includes`
5. `VendorOverride.includes`
6. `SocVendorOverride.includes`
7. `VendorRelease.includes` (sub-release, if `vendor_release` is set)
8. `device.includes`
9. `feature.includes` (for each enabled feature)
10. `feature.VendorOverride.includes`
11. `feature.SocVendorOverride.includes`
12. `feature.VendorRelease.includes`

6.7 Containers

```
containers:
  ubuntu-22.04:
    image: "bsp/kas:5.2"
    file: Dockerfile
    args:
```

```
      - name: "KAS_VERSION"
        value: "5.2"
    runtime_args: "--device=/dev/net/tun --cap-add=NET_ADMIN"
    volumes:
      - host: "$ENV{HOME}/sstate-cache"
        container: "/sstate-cache"
      - host: "/opt/downloads"
        container: "/downloads"
        read_only: true
  isar-debian-13:
    image: "ghcr.io/ilbers/isar:latest"
```

Field	Description
`image`	Docker image used at runtime
`file`	Optional path to Dockerfile for `docker build`
`args`	Docker build arguments (`name`/`value` pairs)
`runtime_args`	Extra flags passed to the container engine (`run`)
`volumes`	List of host-to-container directory mounts (`host`, `container`, `read_only`)

6.8 Named Environments

Named environments bundle a container with environment variables and setup file-copy entries under a single name, referenced by releases:

```
environments:
  default:                    # used when release has no environment
      field
    container: "ubuntu-22.04"
    variables:
      - name: "DL_DIR"
        value: "$ENV{HOME}/data/cache/downloads"
      - name: "SSTATE_DIR"
        value: "$ENV{HOME}/data/cache/sstate"
    copy:
      - scripts/env-setup.sh: build/

  isar-build:                 # named environment for Isar releases
    container: "isar-debian-13"
    variables:
      - name: "DL_DIR"
        value: "$ENV{HOME}/data/cache/isar-downloads"
    copy:
      - isar/scripts/isar-runqemu.sh: build/
```

Container priority: `device.build.container` (highest) > named environment container > none (bare `kas` run).
Variable merge order: root `environment.variables` → named environment variables → feature `env` variables (feature wins).

7. Feature System

Features are optional, composable add-ons mixed into any BSP preset. They support device compatibility constraints, framework/distro restrictions, release-specific includes, and vendor-specific includes.

```yaml
registry:
  features:
    - slug: rauc
      description: "Enable RAUC OTA update support"
      compatible_with: [yocto]    # only compatible with Yocto framework
      compatibility:
        soc_vendor: [nxp]         # allow-list: only NXP SoC devices
      includes:
        - features/ota/rauc/rauc.yml
      local_conf:
        - "DISTRO_FEATURES:append = ' rauc'"
      env:
        - name: "RAUC_KEY_FILE"
          value: "$ENV{RAUC_KEY_FILE}"
      vendor_overrides:
        - vendor: advantech-europe
          includes:
            - features/ota/rauc/advantech-rauc.yml
          soc_vendors:
            - vendor: nxp
              includes:
                - features/ota/rauc/modular-bsp-ota-nxp.yml
              releases:
                - slug: imx-6.6.53
                  includes:
                    - features/ota/rauc/rauc-imx-6.6.53.yml
      release_overrides:
        - release: scarthgap
          includes:
            - features/ota/rauc/rauc-scarthgap.yml
```

7.1 Device Compatibility (`compatibility`)

Key	Meaning
`vendor`	Allow-list of board vendor names (empty = all)
`soc_vendor`	Allow-list of SoC vendor names (empty = all)
`soc_family`	Allow-list of SoC family strings (empty = all)

If any constraint fails, the build exits with a clear error message before any files are modified or processes started.

7.2 Framework/Distro Compatibility (`compatible_with`)

Condition	Result
`compatible_with` is empty	Compatible with all releases
Release's distro slug is in the list	Compatible
Release's distro's framework slug is in the list	Compatible
Neither distro nor framework is in the list	Build exits with error

7.3 Feature Catalogue

Slug	Framework	Description
`systemd`	yocto	Enable systemd as the init system
`yocto-ssh`	yocto	Include SSH server in the image
`debug-tweaks`	yocto	Debug tweaks (empty root password, ...)
`root-login`	yocto	Enable root login
`security`	yocto	Security hardening meta-layer
`secure-boot`	yocto	NXP HAB / AHAB cryptographic signing
`virtualization`	yocto	KVM / container virtualisation
`wayland`	yocto	Wayland display server
`x11`	yocto	X11 display server
`ipv6`	yocto	IPv6 networking
`udev`	yocto	udev device manager
`usrmerge`	yocto	/usr merge (FHS compliance)
`rauc`	yocto	RAUC Over-the-Air update framework
`swupdate`	yocto	SWUpdate OTA framework
`ostree`	yocto	OSTree atomic filesystem upgrades
`hailo`	yocto	Hailo-8 AI accelerator support
`ros2`	yocto	ROS 2 (ros2-humble-scarthgap)
`isar-users`	isar	Sample non-root user accounts (Isar only)

7.4 Release Overrides

A feature can include different KAS files per Yocto/Isar release. The include order for a feature with both `release_overrides` and `vendor_overrides` is:

1. `feature.includes` (always applied)
2. `feature.release_overrides[release].includes` (when release slug matches)
3. `feature.VendorOverride.includes`
4. `feature.SocVendorOverride.includes`
5. `feature.VendorRelease.includes`

8. OTA Update Support

Three OTA technologies are supported as first-class composable features. All major Advantech Europe NXP boards support all three methods.

Technology	Slug	Supported Boards	Releases
RAUC	`rauc`	RSB-3720, ROM-2620, ROM-2820, ROM-5720/21/22	scarthgap – whinlatter
SWUpdate	`swupdate`	RSB-3720, ROM-2620, ROM-2820, ROM-5720/21/22	scarthgap – whinlatter
OSTree	`ostree`	RSB-3720, ROM-2620, ROM-2820, ROM-5720/21/22	scarthgap – whinlatter

```
# Build RSB-3720 with RAUC OTA (Walnascar release)
bsp build modular-bsp-rauc-rsb3720 --release walnascar

# Build RSB-3720 with SWUpdate
bsp build modular-bsp-swupdate-rsb3720 --release scarthgap

# Build RSB-3720 with OSTree
bsp build modular-bsp-ostree-rsb3720 --release walnascar
```

9. Secure Boot

NXP-based Advantech boards support cryptographic boot-image signing via HAB (i.MX8 family) and AHAB (i.MX9/i.MX95 family). The `secure-boot` feature uses the `ubuntu-22.04-csb` container which automatically mounts the NXP Code Signing Tool (CST) and key material from the host.

SoC Family	Technology	Boards
i.MX8	HAB (High Assurance Boot)	RSB-3720, ROM-2620, ROM-5720, ROM-5721, ROM-5722
i.MX93	AHAB (Advanced High Assurance Boot)	ROM-2820
i.MX95	AHAB	AOM-5521

```
# Export signing key environment variables
export CST_TOOL_PATH=/opt/nxp/cst/
export KEYS_PATH=/path/to/srk-keys/

# Build with Secure Boot (ubuntu-22.04-csb container mounts key paths)
bsp build modular-bsp-rom5721-2g-db5901-secureboot --release walnascar
```

10. Containers & Environments

Container	Base OS	KAS	Use Case
ubuntu-20.04	Ubuntu 20.04	4.7	Kirkstone / Mickledore legacy builds
ubuntu-22.04	Ubuntu 22.04	5.2	Default Yocto builds
ubuntu-22.04-csb	Ubuntu 22.04	5.2	Secure Boot builds (key mounts)
ubuntu-24.04	Ubuntu 24.04	5.2	Latest Yocto builds
debian-12	Debian 12	5.2	Hardened / alternative builds
debian-13	Debian 13	5.2	Debian-based Yocto builds
isar-debian-13	Debian 13	5.2	Isar privileged builds

11. CLI Reference

11.1 Build Commands

Command	Description
`bsp build <preset>`	Build a named preset with its default release
`bsp build <preset> -release <r>`	Override the Yocto/Isar release
`bsp build <preset> -checkout`	Validate config only (no actual build)
`bsp build <preset> -deploy`	Build and upload artifacts to cloud storage
`bsp build <preset> -clean`	Clean build directory before building
`bsp build -device <d> -release <r>`	Component-based build (no preset required)
`bsp build -device <d> -release <r> -features f1,f2`	Component-based build with features
`bsp shell <preset>`	Open a shell in the build container
`bsp export <preset>`	Export the resolved KAS YAML configuration

11.2 Listing & Inspection

Command	Description
`bsp list`	List all named BSP presets
`bsp list devices`	List all device slugs
`bsp list releases`	List all release slugs
`bsp list features`	List all feature slugs
`bsp containers`	List container definitions

11.3 Cloud Artifacts

Command	Description
`bsp deploy <preset>`	Upload build artifacts to cloud storage
`bsp deploy <preset> -dry-run`	Preview uploads without sending anything
`bsp gather <preset>`	Download previously uploaded artifacts
`bsp gather <preset> -release <r>`	Download artifacts for a specific release

11.4 Registry Management

Command	Description
`bsp registry init`	Scaffold a new registry YAML file
`bsp registry validate`	Validate registry schema and references
`bsp registry diff <f1> <f2>`	Diff between two registry files
`bsp registry add device ...`	Add a new device entry
`bsp registry add release ...`	Add a new release entry
`bsp registry add feature ...`	Add a new feature entry
`bsp registry edit device ...`	Edit an existing device
`bsp registry remove device <slug>`	Remove a device entry

11.5 Remotes & Server

Command	Description
`bsp remotes add <name> <url>`	Save a named remote registry URL
`bsp remotes show`	List all saved remotes
`bsp remotes remove <name>`	Remove a saved remote
`bsp remotes rename <old> <new>`	Rename a saved remote
`bsp server`	Start REST + GraphQL server
`bsp completions bash|zsh|fish`	Print shell completion script

11.6 Global CLI Flags

Flag	Description
`-registry <path>`	Use an explicit local registry YAML
`-local`	Use `./bsp-registry.yml`, no network
`-remote <url>`	Remote URL or saved remote name
`-branch <b>`	Git branch for `-remote`
`-no-update`	Skip git-pull of the remote registry

11.7 Quick Start

```
# First run: auto-clones the Advantech registry
bsp list
```

```
# Build with default release
bsp build modular-bsp-rsb3720

# Build with a specific Yocto release
bsp build modular-bsp-rsb3720 --release walnascar

# Validate config without building
bsp build modular-bsp-rsb3720 --checkout

# Component-based build (no preset required)
bsp build --device rsb3720 --release scarthgap --features systemd,
    security

# Use the Advantech development branch
bsp --remote https://github.com/Advantech-EECC/bsp-registry.
    git@development list

# Save a remote for reuse
bsp remotes add advantech https://github.com/Advantech-EECC/bsp-registry.
    git
bsp --remote advantech list
```

12. Python API

All CLI operations are backed by a public Python API. The primary entry point is `BspManager`, which coordinates registry loading, preset resolution, builds, deployment, and artifact gathering.

12.1 Basic Usage

```
from bsp import BspManager, RegistryFetcher, V2Resolver

# 1. Fetch / update the remote registry
fetcher = RegistryFetcher()
registry_path = fetcher.fetch_registry(
    repo_url="https://github.com/Advantech-EECC/bsp-registry.git",
    branch="development",
    update=True,
)

# 2. Load registry and inspect contents
manager = BspManager(str(registry_path))
manager.initialize()

for dev in manager.model.registry.devices:
    print(dev.slug, "-", dev.description)

for rel in manager.model.registry.releases:
    print(rel.slug, rel.yocto_version)
```

12.2 Building and Deploying

```
from bsp import BspManager
```

```
manager = BspManager("bsp-registry.yaml")
manager.initialize()

# Build a named preset
result = manager.build_bsp("modular-bsp-rsb3720", release="walnascar")
print("Build exited with:", result.returncode)

# Deploy artifacts to cloud storage
deploy_result = manager.deploy_bsp("modular-bsp-rsb3720")
for artifact in deploy_result.uploaded:
    print("Uploaded:", artifact.name, "->", artifact.url)

# Gather (download) previously uploaded artifacts
gather_result = manager.gather_bsp("modular-bsp-rsb3720", release="
   walnascar")
for path in gather_result.downloaded:
    print("Downloaded:", path)
```

12.3 Resolving a Configuration

```
from bsp import BspManager, V2Resolver

manager = BspManager("bsp-registry.yaml")
manager.initialize()

resolver = V2Resolver(manager.model)
config = resolver.resolve(
    device_slug="rsb3720",
    release_slug="walnascar",
    feature_slugs=["systemd", "security", "rauc"],
)

# Inspect the resolved KAS file list
for f in config.kas_files:
    print(f)

# Inspect environment variables
for var in config.env_vars:
    print(f"{var.name}={var.value}")
```

12.4 Multi-Registry Mode

```
from bsp import BspManager

manager = BspManager(
    config_paths=[
        ("advantech",    "/path/to/advantech-registry.yaml"),
        ("my-additions", "/path/to/my-registry.yaml"),
    ]
)
manager.initialize()

# Disambiguate with registry:preset syntax
result = manager.build_bsp("advantech:modular-bsp-rsb3720")
```

13. Cloud Artifact Deployment

13.1 Registry Configuration

```
specification:
  version: "2.0"

deploy:
  provider: azure
  account_url: $ENV{AZURE_STORAGE_ACCOUNT_URL}
  container: bsp-artifacts
  prefix: "{vendor}/{device}/{release}/{date}"
  patterns:
    - "**/*.wic.gz"
    - "**/*.wic.bz2"
    - "**/*.tar.bz2"
    - "**/*.ext4"
```

Prefix Placeholder	Expands to
{vendor}	Board vendor slug
{device}	Device slug
{release}	Release slug
{date}	Build date (YYYY-MM-DD)
{preset}	Preset name

13.2 Authentication

```
# Azure (DefaultAzureCredential chain: CLI, managed identity, ...)
az login
export AZURE_STORAGE_ACCOUNT_URL="https://myaccount.blob.core.windows.net
    "

# AWS
aws configure
# or
export AWS_ACCESS_KEY_ID="..."
export AWS_SECRET_ACCESS_KEY="..."
export AWS_DEFAULT_REGION="eu-west-1"
```

13.3 CLI Deploy / Gather

```
# Build and deploy in one step
bsp build modular-bsp-rsb3720 --release walnascar --deploy

# Deploy artifacts from an existing build
bsp deploy modular-bsp-rsb3720 --container bsp-artifacts

# Preview what would be uploaded (no credentials required)
bsp deploy modular-bsp-rsb3720 --dry-run

# Download previously uploaded artifacts
```

```
bsp gather modular-bsp-rsb3720 --release walnascar
```

14. HTTP Server (REST + GraphQL)

```
pip install "bsp-registry-tools[server]"

# Default: http://127.0.0.1:8080
bsp server

# Bind to all interfaces, custom port, auto-reload for development
bsp server --host 0.0.0.0 --port 9000 --reload

# Use a specific registry
bsp --registry /path/to/bsp-registry.yaml server --host 0.0.0.0
```

14.1 REST API Endpoints

Method	Path	Description
GET	/api/v1/devices	List all devices
GET	/api/v1/releases	List all releases
GET	/api/v1/features	List all features
GET	/api/v1/bsp	List all named presets
POST	/api/v1/bsp/{name}/build	Trigger a build
POST	/graphql	GraphQL endpoint
GET	/docs	OpenAPI (Swagger) docs

14.2 GraphQL Query Example

```
curl -X POST http://localhost:8080/graphql \
  -H 'Content-Type: application/json' \
  -d '{"query": "{ releases { slug description yoctoVersion } }"}'
```

14.3 Starting the Server from Python

```
import uvicorn
from bsp import BspManager
from bsp.server import create_app

# Reuse an already-initialised manager (avoids double registry load)
manager = BspManager("bsp-registry.yaml")
manager.initialize()

app = create_app(manager=manager)
uvicorn.run(app, host="0.0.0.0", port=8080)
```

15. CI/CD Integration

15.1 GitHub Actions — Azure

```yaml
# .github/workflows/build.yml
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Install bsp-registry-tools
        run: pip install "bsp-registry-tools[azure]"

      - name: Validate registry
        run: bsp --local registry validate

      - name: Fast checkout gate (no full build)
        run: bsp --no-update build modular-bsp-rsb3720 --checkout

      - name: Build and deploy artifacts
        env:
          AZURE_STORAGE_ACCOUNT_URL: ${{ secrets.
              AZURE_STORAGE_ACCOUNT_URL }}
        run: |
          bsp --no-update build modular-bsp-rsb3720 \
            --release walnascar --deploy
```

15.2 GitHub Actions — AWS

```yaml
      - name: Build and deploy to S3
        env:
          AWS_ACCESS_KEY_ID: ${{ secrets.AWS_ACCESS_KEY_ID }}
          AWS_SECRET_ACCESS_KEY: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          AWS_DEFAULT_REGION: eu-west-1
        run: |
          pip install "bsp-registry-tools[aws]"
          bsp --no-update build modular-bsp-rsb3720 \
            --release walnascar --deploy
```

16. Extending BSPs with Custom Yocto Layers

Export a resolved KAS configuration and include it in a downstream project to add custom meta-layers without modifying the registry:

```
# Export the resolved KAS config for a specific preset + release
bsp export modular-bsp-rsb3720 --release walnascar \
  > modular-bsp-rsb3720-walnascar.yaml
```

```yaml
# my-custom-kas.yaml
header:
  version: 19
```

```
  includes:
    - repo: bsp-registry
      file: modular-bsp-rsb3720-walnascar.yaml  # from bsp export

repos:
  bsp-registry:
    url: "https://github.com/Advantech-EECC/bsp-registry"
    branch: "development"
    layers: { .: "disabled" }

  meta-custom:
    layers:
      meta-custom:    # your custom Yocto meta-layer
```

```
# meta-custom/recipes-core/images/imx-image-%.bbappend
# CORE_IMAGE_EXTRA_INSTALL += "mpv"
# LICENSE_FLAGS_ACCEPTED += "commercial"

# Build with your extension
kas build my-custom-kas.yaml
```

17. Summary

Feature	Benefit
YAML registry v2	Single source of truth for 30+ boards × 8+ releases
Auto remote fetch	Zero-config first run; teams share the Advantech registry
Two build systems	Yocto/BitBake AND Isar/Debian in one registry
KAS integration	Reproducible builds for every device × release combo
Vendor overrides	NXP/MediaTek/Qualcomm-specific KAS fragments, distros, kernel pins
Named environments	Correct container per build class (secure-boot, Isar, ...)
Feature system	Composable add-ons with compat. checks, release + vendor overrides
OTA support	RAUC, SWUpdate, OSTree — all first-class features
Secure Boot	NXP HAB / AHAB signing built into the registry
Cloud deploy / gather	Full artifact lifecycle: build → upload → download
HTTP server	REST + GraphQL for dashboards and automation
Tab completions	Context-aware completions for all CLI arguments
Python API	Integrate into custom tools, CI scripts, and dashboards
Registry editor	CRUD for devices, releases, features, presets with validation

Registry: https://github.com/Advantech-EECC/bsp-registry (development branch)